

DAT	ORG		
	MOV	i,#9	'set i to 9
loop	DEBUG	(UHEX_LONG(i))	'debug, output i in hex
	DJNF	i,#loop	'decrement i and loop if not -1
	JMP	#\$	'don't go wandering off, stay here
i	RES	1	'reserve one register as 'i'

When run with Ctrl-F10, the Debug window opens and this is what appears:

```
Cog0  INIT $0000_0000 $0000_0000 load
Cog0  i = $0000_0009
Cog0  i = $0000_0008
Cog0  i = $0000_0007
Cog0  i = $0000_0006
Cog0  i = $0000_0005
Cog0  i = $0000_0004
Cog0  i = $0000_0003
Cog0  i = $0000_0002
Cog0  i = $0000_0001
Cog0  i = $0000_0000
```

In the first line of the report, you see Cog0 loading our PASM program from \$000000. After that, the program runs and you see 'i' iterating from 9 down to 0.

If you change the "9" to "99" in the MOV instruction and you'd like to slow things down, add a DLY command to the DEBUG statement and be sure to express the milliseconds as #250, since a plain 250 would be understood as register 250:

```
debug    (uhex_long(i), dly(#250))    'debug, output i in hex and delay for 250ms after each report
```

DEBUG memory utilization

Here is what the memory utilization looks like for a Spin2 DEBUG statement. You can see, on the Spin2 side, that a bytecode is needed to read the variable 'i', and then three obligatory bytecodes make up the actual DEBUG instruction.

The 'stack adjustment' byte tells the interpreter how far to drop the stack to effectively 'pop' all the expressions that were pushed in preparation for the DEBUG event. In this case of 'i', only, the stack needs to drop by four bytes (one long). When the debugger is invoked, the values it needs will be ordered right above the current Spin2 stack pointer.

```
debug("What? ", udec(i))
```

Spin2 bytecodes in application

```
$E0 - read 'i'
$44 - DEBUG bytecode
$04 - stack adjustment
$01 - unique BRK code
```

DEBUG database in
top 16KB of RAM

```
$04 - output "CogN  "  
$06 - output string  
$57 - "W"  
$68 - "h"  
$61 - "a"  
$74 - "t"  
$3F - "?"  
$20 - " "  
$00 - end of string  
$41 - UDEC + output string  
$69 - "i"  
$00 - end of string  
$00 - end of DEBUG statement
```

The 'unique BRK code' byte is used as an index to look up the specific record in the DEBUG database at the top of memory, from which the debugger reads its commands.

In the case where DEBUG is active, but a cog has had its debug interrupt disabled via the `DEBUG_ENABLE` symbol, Spin2 DEBUG instructions will not trigger a debug interrupt, but they do still pop any DEBUG-intended values from the stack, so these are harmless events.

For PASM DEBUG statements, a 'BRK #code' instruction is inserted where the DEBUG command was placed, and all related data resides in the DEBUG database. If the cog's debug interrupt is disabled, the 'BRK #code' instruction does nothing, taking two clocks.

DEBUG and interrupts

Interrupt requests received during a DEBUG statement will execute after the DEBUG completes, but the response time may be so skewed that the retrigger setup for the interrupt won't happen properly. High-frequency cyclical smart pin interrupts are especially prone to this problem. Imagine you do an AKPIN instruction within your normal ISR (interrupt service routine) to drop the INA/INB signal so that the smart pin can make it go high again, triggering a new interrupt. Meanwhile, after the AKPIN and before the RETIx, the smart pin triggers, raising INA/INB high. This is only happening because your cycle-frame timing has become skewed from the DEBUG statement. This interrupt won't be seen since it happened when the ISR was busy. This will cause the interrupt to cease cycling. CT interrupts are not prone to this problem, though, since they have \$8000_0000 clock cycles in which to be recognized. To remedy the smart-pin retrigger problem, you could trigger on INA/INB-high, as opposed to INA/INB-rise, but this could cause performance problems with your smart pin configurations.

One fail-safe way to get around this DEBUG/interrupt dilemma is to only do DEBUG statements from cogs that are not executing ISRs in the background. If the ISRs can tolerate timing skew and there is no risk of hanging interrupt cycling, you can do DEBUG statements with some understood interrupt timing degradations.

Graphical DEBUG Displays

DEBUG messages can invoke special graphical DEBUG displays which are built into the tool. These graphical displays each take the form of a unique window. Once instantiated, displays can be continuously fed data to generate animated visualizations. These displays are very handy for development and debugging, as various data types can be viewed in their native contexts. Up to 32 graphical displays can be running simultaneously.

When a DEBUG message contains a backtick (`) character (ASCII \$60), a string, containing everything from the backtick to the end of the message, is sent to the graphical DEBUG display parser. The parser looks for several different element types, treating any commas as whitespace:

Element Type	Example	Description
display_type	LOGIC, SCOPE, PLOT, BITMAP	This is the formal name of the graphical DEBUG display type you wish to instantiate.
unknown_symbol	MyLogicDisplay	Each graphical DEBUG display Instance must be given a unique symbolic name.
instance_name	MyLogicDisplay	Once instantiated, a graphical DEBUG display instance is referenced by its symbolic name.
keyword	TITLE, POS, SIZE, SAMPLES	Keywords are used to configure displays. They might be followed by numbers, strings, and other keywords.
number	1024, \$FF, %1010	Numbers can be expressed in decimal, hex (\$), and binary (%).
string	'Here is a string'	Strings are expressed within single-quotes.

Before getting into how all this fits together, we need to go over some special DEBUG-display syntax that can be used for displays. This syntax is invoked when the first character in the DEBUG statement is the backtick. This causes everything in the DEBUG statement to be viewed as a string, except when subsequent backticks act as 'escape' characters to allow normal or shorthand DEBUG commands.

DEBUG Statement (v=100, BYTE[a]=1,2,3,4,5)	DEBUG Message Output	Note
DEBUG("`LOGIC MyDisplay SAMPLES ", SDEC(v))	Cog0 `LOGIC MyDisplay SAMPLES 100	Regular DEBUG syntax can drive DEBUG displays, but it's not optimal.
DEBUG(`LOGIC MyDisplay SAMPLES 100)	`LOGIC MyDisplay SAMPLES 100	DEBUG-display syntax is simpler and 'CogN' is omitted in the output.
DEBUG(`LOGIC MyDisplay SAMPLES `(v))	`LOGIC MyDisplay SAMPLES 100	Decimal numbers are output using `(value) notation. Short for SDEC_.
DEBUG(`LOGIC MyDisplay SAMPLES `\${v})	`LOGIC MyDisplay SAMPLES \$64	Hex numbers are output using `\${value) notation. Short for UHEX_.
DEBUG(`LOGIC MyDisplay SAMPLES `%v))	`LOGIC MyDisplay SAMPLES %1100100	Binary numbers are output using `%value) notation. Short for UBIN_.
DEBUG(`LOGIC MyDisplay TITLE `'#(v)')	`LOGIC MyDisplay TITLE 'd'	Characters are output using `#value) notation.
DEBUG(`MyDisplay `UDEC_BYTE_ARRAY_(@a,5))	`MyDisplay 1, 2, 3, 4, 5	Regular DEBUG commands can follow the backtick, as well.

There are two steps to using graphical DEBUG displays. First, they must be instantiated and, second, they must be fed:

To Use a Display:	1st	2nd	3rd	4th	Note
First, instantiate it.	`	display_type	unknown_symbol	keyword(s), number(s), string(s)	Unknown_symbol becomes instance_name.
Then, feed it.	`	instance_name(s)	keyword(s), number(s), string(s)		Multiple displays can be fed the same data.

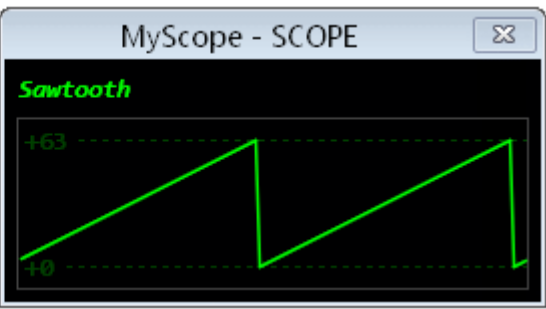
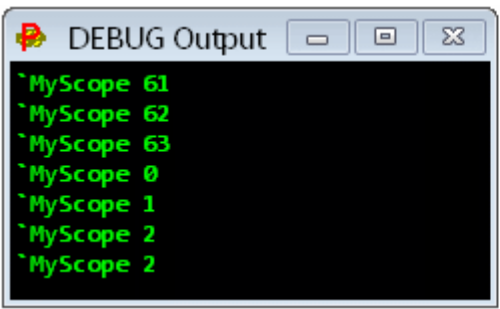
To bring this all together, let's show a sawtooth wave on a SCOPE display:

```
CON _clkfreq = 10_000_000

PUB go() | i

  debug(`SCOPE MyScope SIZE 254 84 SAMPLES 128)
  debug(`MyScope 'Sawtooth' 0 63 64 10 %1111)

  repeat
    debug(`MyScope `(i & 63))
    i++
    waitms(50)
```



In the example above, a SCOPE is instantiated called MyScope that is 254 x 84 pixels and shows 128 samples. A width of 254 was chosen since samples are numbered 0..127 and I wanted them to be spaced at a constant two-pixel pitch (127 * 2 = 254). A height of 84 was chosen so that there would be 10 pixels above and below the waveform, which will have a height of 64 pixels. A channel called "Sawtooth" is defined which, for the purpose of display, has a bottom value of 0 and a top value of 63, is 64 pixels tall within that range, and is elevated 10 pixels off the bottom of the scope window. The %1111 enables top and bottom legend values and top and bottom lines. Within the REPEAT block, the SCOPE is fed a repeating pattern of 0..63 which forms the sawtooth wave. The SCOPE updates its display each time it receives a value. If there were eight channels defined, instead of just one, it would update the display on every eighth value received, drawing all eight channels.

Currently, the following graphical DEBUG displays are implemented, but more will be added in the future:

Display Types	Descriptions
LOGIC	Logic analyzer with single and multi-bit labels, 1..32 channels, can trigger on pattern
SCOPE	Oscilloscope with 1..8 channels, can trigger on level with hysteresis
SCOPE_XY	XY oscilloscope with 1..8 channels, persistence of 0..512 samples, polar mode, log scale mode
FFT	Fast Fourier Transform with 1..8 channels, 4..2048 points, windowed results, log scale mode
SPECTRO	Spectrograph with 4..2048-point FFT, windowed results, phase-coloring, and log scale mode
PLOT	General-purpose plotter with cartesian and polar modes
TERM	Text terminal with up to 300 x 200 characters, 6..200 point font size, 4 simultaneous color schemes
BITMAP	Bitmap, 1..2048 x 1..2048 pixels, 1/2/4/8/16/32-bit pixels with 19 color systems, 15 direction/autoscroll modes, independent X and Y pixel size of 1..256
MIDI	Piano keyboard with 1..128 keys, velocity depiction, variable screen scale

Following are elaborations of each DEBUG display type.

LOGIC Display

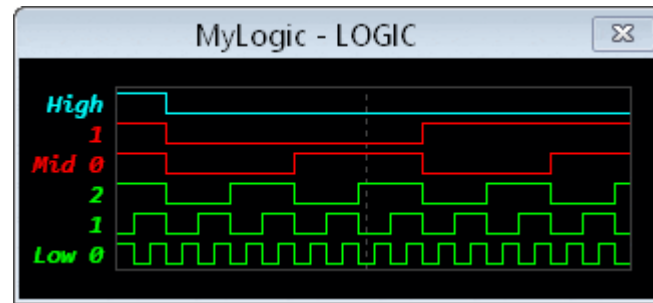
Logic analyzer with single and multi-bit labels, 1..32 channels, can trigger on pattern

```
CON _clkfreq = 10_000_000

PUB go() | i

  debug(`LOGIC MyLogic SAMPLES 32 'Low' 3 'Mid' 2 'High')
  debug(`MyLogic TRIGGER $07 $04 HOLDOFF 2)

  repeat
    debug(`MyLogic `(i & 63))
    i++
    waitms(25)
```



LOGIC Instantiation	Description	Default
TITLE 'string'	Set the window caption to 'string'.	<none>
POS left top	Set the window position.	0, 0
SAMPLES 4_to_2048	Set the number of samples to track and display.	32
SPACING 2_to_32	Set the sample spacing. The width of the display will be SAMPLES * SPACING.	8
RATE 1_to_2048	Set the number of samples (or triggers, if enabled) before each display update.	1
LINESIZE 1_to_7	Set the line size.	1
TEXTSIZE 6_to_200	Set the legend text size. Height of text determines height of logic levels.	editor text size
COLOR back_color {grid_color}	Set the background and grid colors *.	BLACK, GREY 4
'name' {1_to_32 {color}}	Set the first/next channel or group name, optional bit count, optional color *.	1, default color
packed_data_mode	Enable packed-data mode. See description at end of this section.	<none>
LOGIC Feeding	Description	Default
TRIGGER mask match sample_offset	Trigger on (data & mask) = match. If mask = 0, trigger is disabled.	0, 1, SAMPLES / 2
HOLDOFF 2_to_2048	Set the minimum number of samples required from trigger to trigger.	SAMPLES
data	Numerical data is applied LSB-first to the channels.	
CLEAR	Clear the sample buffer and display, wait for new data.	
SAVE {WINDOW} 'filename'	Save a bitmap file (.bmp) of either the entire window or just the display area.	
CLOSE	Close the window.	

* Color is rgb24 value, else BLACK / WHITE or ORANGE / BLUE / GREEN / CYAN / RED / MAGENTA / YELLOW / GREY followed by an optional 0..15 for brightness (default is 8).

The LOGIC display can be used to display data that was captured at high speed. In the example below, the P2 is generating 8-N-1 serial at 333 Mbaud using a smart pin. This bit stream can be captured by the streamer. On every clock, the streamer will record the smart pin's IN signal and its output state, as read from an adjacent pin. Every time it gets four two-bit sample sets, it does an RFBYTE to save them to hub RAM, forming contiguous bytes, words, and longs. By invoking the LONGS_2BIT packed-data mode, we can have the LOGIC display unpack the two-bit sample sets from longs, yielding 16 sets per long.

```
CON _clkfreq = 333_333_333 'go really fast, 3ns clock period
rxpin = 24 'even pin
txpin = rxpin+1 'odd pin
samps = 32 'multiple of 16 samples
bufflongs = samps / 16 'each long holds 16 2-bit samples
xmode = $D0800000 + rxpin << 17 + samps 'streamer mode

VAR buff[bufflongs]

PUB go() | i, buffaddr

  debug(`logic Serial samples `(samps) spacing 12 'TX' 'IN' longs_2bit)
  debug(`Serial trigger %10 %10 22)
  buffaddr := @buff

  repeat
    org
    wrpin ##+1<<28,#rxpin 'rxpin inputs txpin at rxpin+1

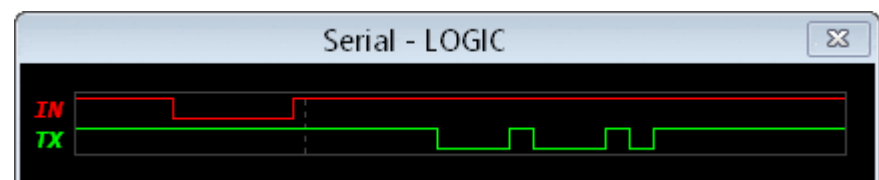
    wrpin #%01_11110_0,#txpin 'set async tx mode for txpin
    wxpin ##1<<16+8-1,#txpin 'set baud=sysclock/1 and size=8
    dirh #txpin 'enable smart pin

    wrfast #0,buffaddr 'set write-fast at buff
    xinit ##xmode,#0 'start capturing 2-bit samples

    wy pin i,#txpin 'transmit serial byte

    waitxfi 'wait for streamer capture done
  end

  debug(`Serial `uhex_long_array_(@buff, bufflongs))
  i++
  waitms(20)
```



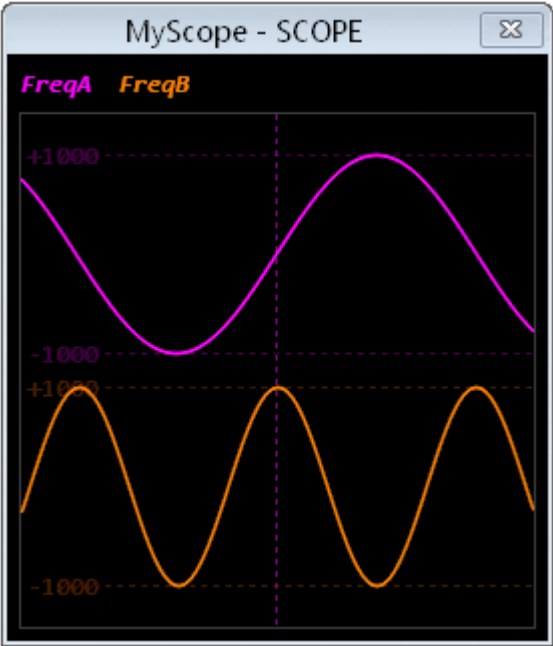
SCOPE Display Oscilloscope with 1..8 channels, can trigger on level with hysteresis

```
CON _clkfreq  = 100_000_000

PUB go() | a, af, b, bf

  debug(`SCOPE MyScope)
  debug(`MyScope 'FreqA' -1000 1000 100 136 15 MAGENTA)
  debug(`MyScope 'FreqB' -1000 1000 100 20 15 ORANGE)
  debug(`MyScope TRIGGER 0 HOLDOFF 2)

  repeat
    a := qsin(1000, af++, 200)
    b := qsin(1000, bf++, 99)
    debug(`MyScope `(a,b))
    waitus(200)
```



SCOPE Instantiation	Description	Default
TITLE 'string'	Set the window caption to 'string'.	<none>
POS left top	Set the window position.	0, 0
SIZE width height	Set the display size (32..2048 x 32..2048)	255, 256
SAMPLES 16_to_2048	Set the number of samples to track and display.	256
RATE 1_to_2048	Set the number of samples (or triggers, if enabled) before each display update.	1
DOTSIZE 0_to_32	Set the dot size in pixels for showing exact sample points.	0
LINESIZE 0_to_32	Set the line size in half-pixels for connecting sample points.	3
TEXTSIZE 6_to_200	Set the legend text size.	editor text size
COLOR back_color {grid_color}	Set the background and grid colors *.	BLACK, GREY 4
packed_data_mode	Enable packed-data mode. See description at end of this section.	<none>
SCOPE Feeding	Description	Default
'name' {min {max {y_size {y_base {legend {color}}}}}}	Set first/next channel name, min value, max value, y size, y base, legend, and color *. Legend is %abcd, where %a to %d enable max legend, min legend, max line, min line.	full, no legend, default color
TRIGGER channel {arm_level {trigger_level {offset}}}	Set the trigger channel, arm level, trigger level, and right offset. If channel=-1, disabled.	-1, -1, 0, width / 2
HOLDOFF 2_to_2048	Set the minimum number of samples required from trigger to trigger.	SAMPLES
data	Numerical data is applied to the channels in ascending order.	
CLEAR	Clear the sample buffer and display, wait for new data.	
SAVE {WINDOW} 'filename'	Save a bitmap file (.bmp) of either the entire window or just the display area.	
CLOSE	Close the window.	

* Color is rgb24 value, else BLACK / WHITE or ORANGE / BLUE / GREEN / CYAN / RED / MAGENTA / YELLOW / GREY followed by an optional 0..15 for brightness (default is 8).

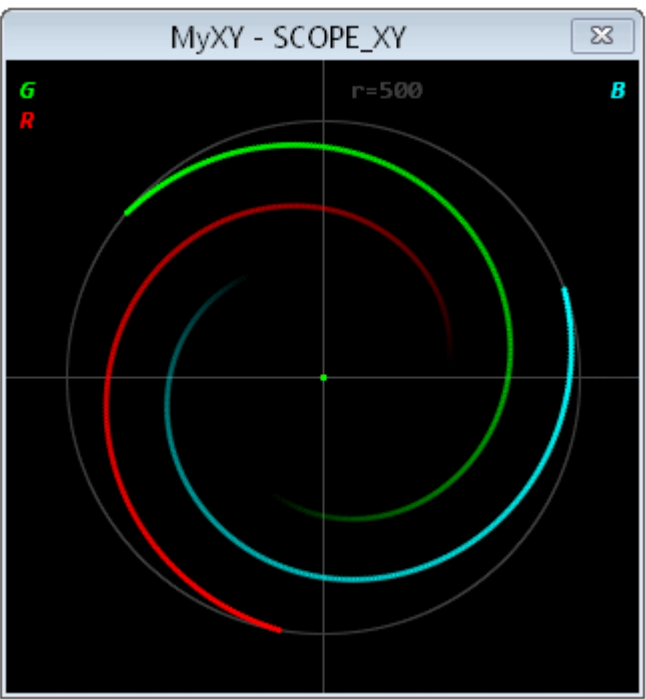
SCOPE_XY Display XY oscilloscope with 1..8 channels, persistence of 1..512 samples, polar mode, log scale mode

```
CON _clkfreq = 100_000_000

PUB go() | i

  debug(`SCOPE_XY MyXY RANGE 500 POLAR 360 'G' 'R' 'B')

  repeat
    repeat i from 0 to 500
      debug(`MyXY `(i, i, i, i+120, i, i+240))
      waitms(5)
```



SCOPE_XY Instantiation	Description	Default
TITLE 'string'	Set the window caption to 'string'.	<none>
POS left top	Set the window position.	0, 0
SIZE radius	Set the display radius in pixels.	128
RANGE 1_to_7FFFFFFF	Set the unit circle radius for incoming data	\$7FFFFFFF
SAMPLES 0_to_512	Set the number of samples to track and display with persistence. Use 0 for infinite persistence.	256
RATE 1_to_512	Set the number of samples before each display update.	1
DOTSIZE 2_to_20	Set the dot size in half-pixels for showing sample points.	6
TEXTSIZE 6_to_200	Set the legend text size.	editor text size
COLOR back_color {grid_color}	Set the background and grid colors *.	BLACK, GREY 4
POLAR {twopi {offset}}	Set polar mode, twopi value, and offset. For a twopi value of \$100000000 or -\$100000000, use 0 or -1.	\$100000000, 0
LOGSCALE	Set log-scale mode to magnify points within the unit circle.	<off>
'name' {color}	Set the first/next channel name and optionally assign it a color *.	default color
packed_data_mode	Enable packed-data mode. See description at end of this section.	<none>
SCOPE_XY Feeding	Description	Default
x y	X-Y data pairs are applied to the channels in ascending order. In polar mode, x=length and y=angle.	
CLEAR	Clear the sample buffer and display, wait for new data.	
SAVE {WINDOW} 'filename'	Save a bitmap file (.bmp) of either the entire window or just the display area.	
CLOSE	Close the window.	

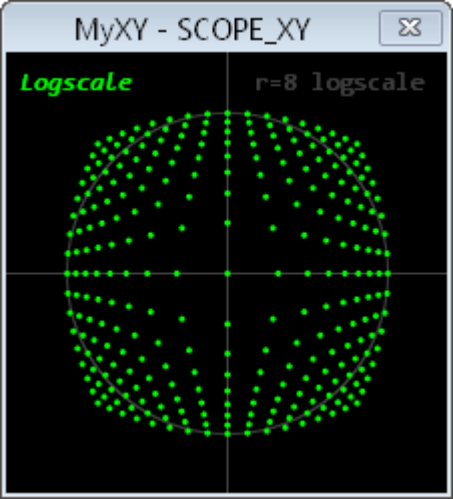
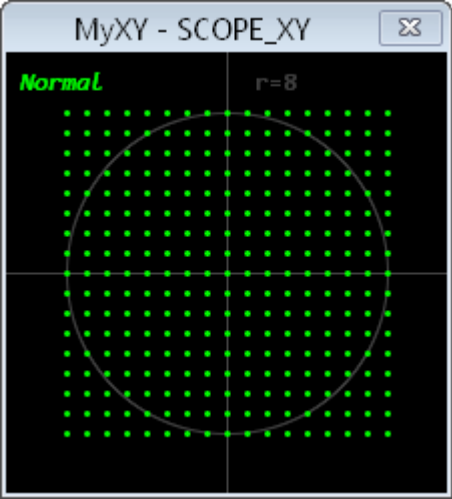
* Color is rgb24 value, else BLACK / WHITE or ORANGE / BLUE / GREEN / CYAN / RED / MAGENTA / YELLOW / GREY followed by an optional 0..15 for brightness (default is 8).

```
CON _clkfreq = 10_000_000      'Normal mode

PUB go() | x, y
  debug(`SCOPE_XY MyXY SIZE 80 RANGE 8 SAMPLES 0 'Normal')
  repeat x from -8 to 8
    repeat y from -8 to 8
      debug(`MyXY `(x,y))
```

```
CON _clkfreq = 10_000_000      'LOGSCALE mode magnifies low-level details

PUB go() | x, y
  debug(`SCOPE_XY MyXY SIZE 80 RANGE 8 SAMPLES 0 LOGSCALE 'Logscale')
  repeat x from -8 to 8
    repeat y from -8 to 8
      debug(`MyXY `(x,y))
```



FFT Display

Fast Fourier Transform with 1..8 channels, 4..2048 points, windowed results, log scale mode

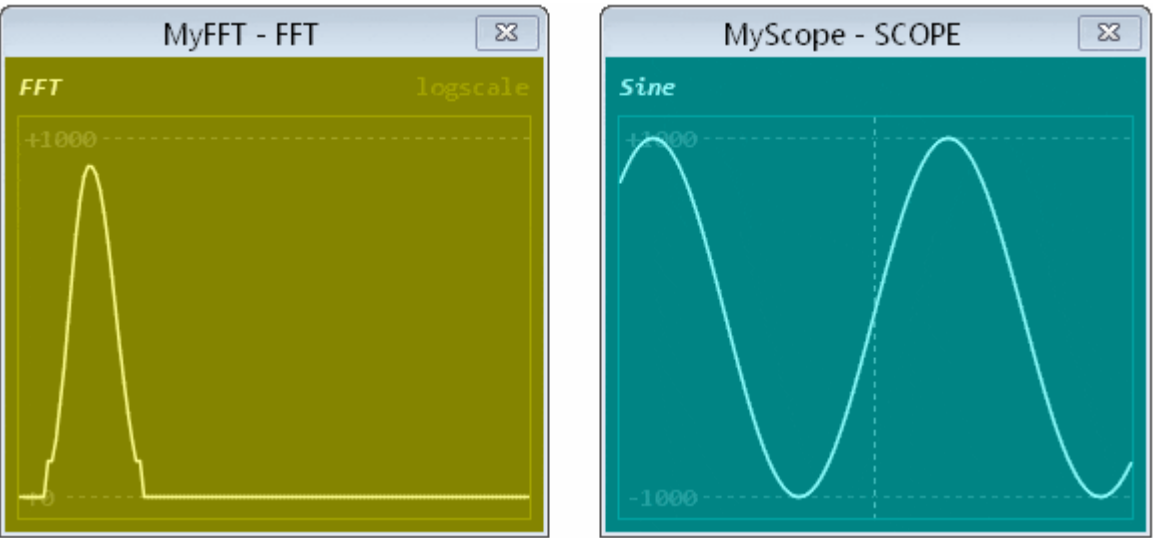
```
CON _clkfreq = 100_000_000

PUB go() | i, j, k

' Set up FFT
debug(`FFT MyFFT SIZE 250 200 SAMPLES 2048 0 127 RATE 256 LOGSCALE COLOR YELLOW 4 YELLOW 5)
debug(`MyFFT 'FFT' 0 1000 180 10 15 YELLOW 12)

' Set up SCOPE
debug(`scope MyScope POS 300 0 SIZE 255 200 COLOR CYAN 4 CYAN 5)
debug(`MyScope 'Sine' -1000 1000 180 10 15 CYAN 12)
debug(`MyScope TRIGGER 0)

repeat
  j += 1550 + qsin(1300, i++, 31_000)
  k := qsin(1000, j, 50_000)
  debug(`MyFFT MyScope `(k))
  waitus(100)
```



FFT Instantiation	Description	Default
TITLE 'string'	Set the window caption to 'string'.	<none>
POS left top	Set the window position.	0, 0
SIZE width height	Set the display size (32..2048 x 32..2048)	256, 256
SAMPLES 4_to_2048 {first {last}}	Set the 2 ⁿ number of FFT inputs points, plus the first and last result values to display.	512, 0, 255
RATE 1_to_2048	Set the number of samples before each display update.	SAMPLES
DOTSIZE 0_to_32	Set the dot size in pixels for showing exact sample points.	0
LINESIZE neg32_to_32	Set the line size in half-pixels for connecting sample points. A negative line size will make isolated vertical lines.	3
TEXTSIZE 6_to_200	Set the legend text size.	editor text size
COLOR back_color {grid_color}	Set the background and grid colors *.	BLACK, GREY 4
LOGSCALE	Set log-scale mode to magnify low-level results.	<off>
packed_data_mode	Enable packed-data mode. See description at end of this section.	<none>
FFT Feeding	Description	Default
'name' {mag {max {y_size {y_base {legend {color}}}}}}	Set the first/next channel name, magnification factor (2 ⁿ , n = 0..11), max amplitude, y size, y base, legend, and color *. Legend is %abcd, where %a to %d enable max legend, min legend, max line, min line.	full, no legend, default color
data	Numerical data is fed into the channels' sliding Hanning windows from which the FFT computes power levels.	
CLEAR	Clear the sample buffer and display, wait for new data.	
SAVE {WINDOW} 'filename'	Save a bitmap file (.bmp) of either the entire window or just the display area.	
CLOSE	Close the window.	

* Color is rgb24 value, else BLACK / WHITE or ORANGE / BLUE / GREEN / CYAN / RED / MAGENTA / YELLOW / GREY followed by an optional 0..15 for brightness (default is 8).

SPECTRO Display Spectrograph with 4..2048-point FFT, phase-coloring, and log scale mode

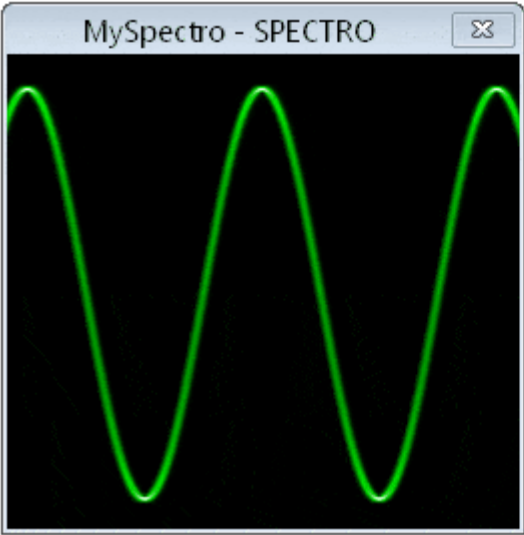
```
CON _clkfreq = 100_000_000

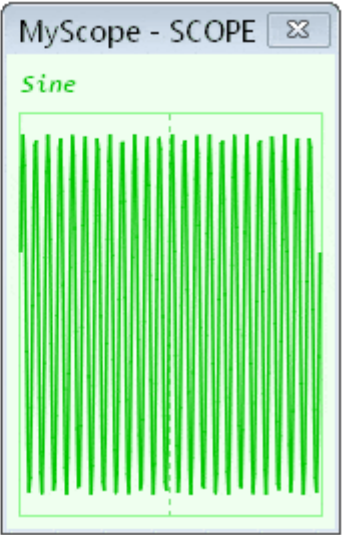
PUB go() | i, j, k

  ' Set up SPECTRO
  debug(`SPECTRO MySpectro SAMPLES 2048 0 236 RANGE 1000 LUMA8X GREEN)

  ' Set up SCOPE
  debug(`SCOPE MyScope POS 280 SIZE 150 200 COLOR GREEN 15 GREEN 12)
  debug(`MyScope 'Sine' -1000 1000 180 10 0 GREEN 6)
  debug(`MyScope TRIGGER 0)

  repeat
    j += 2850 + qsin(2500, i++, 30_000)
    k := qsin(1000, j, 50_000)
    debug(`MySpectro MyScope `(k))
    waitus(100)
```

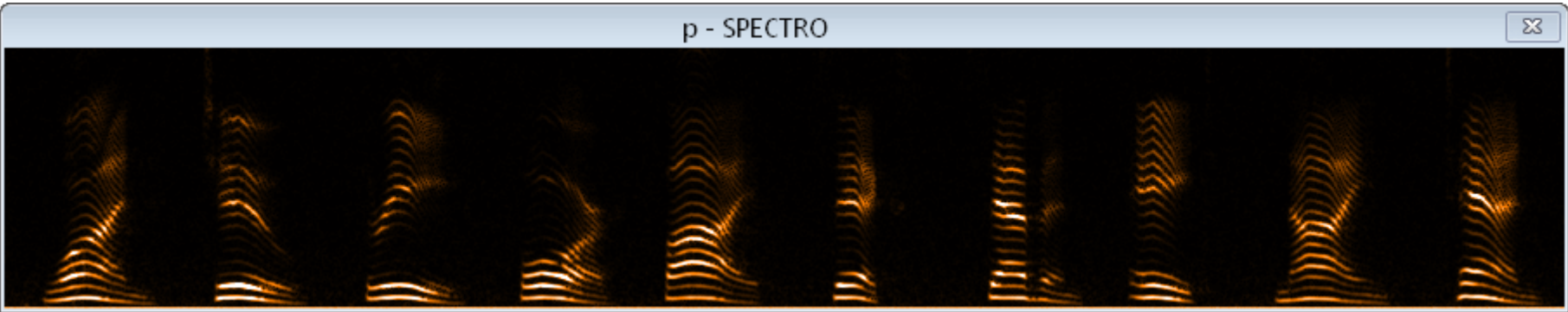


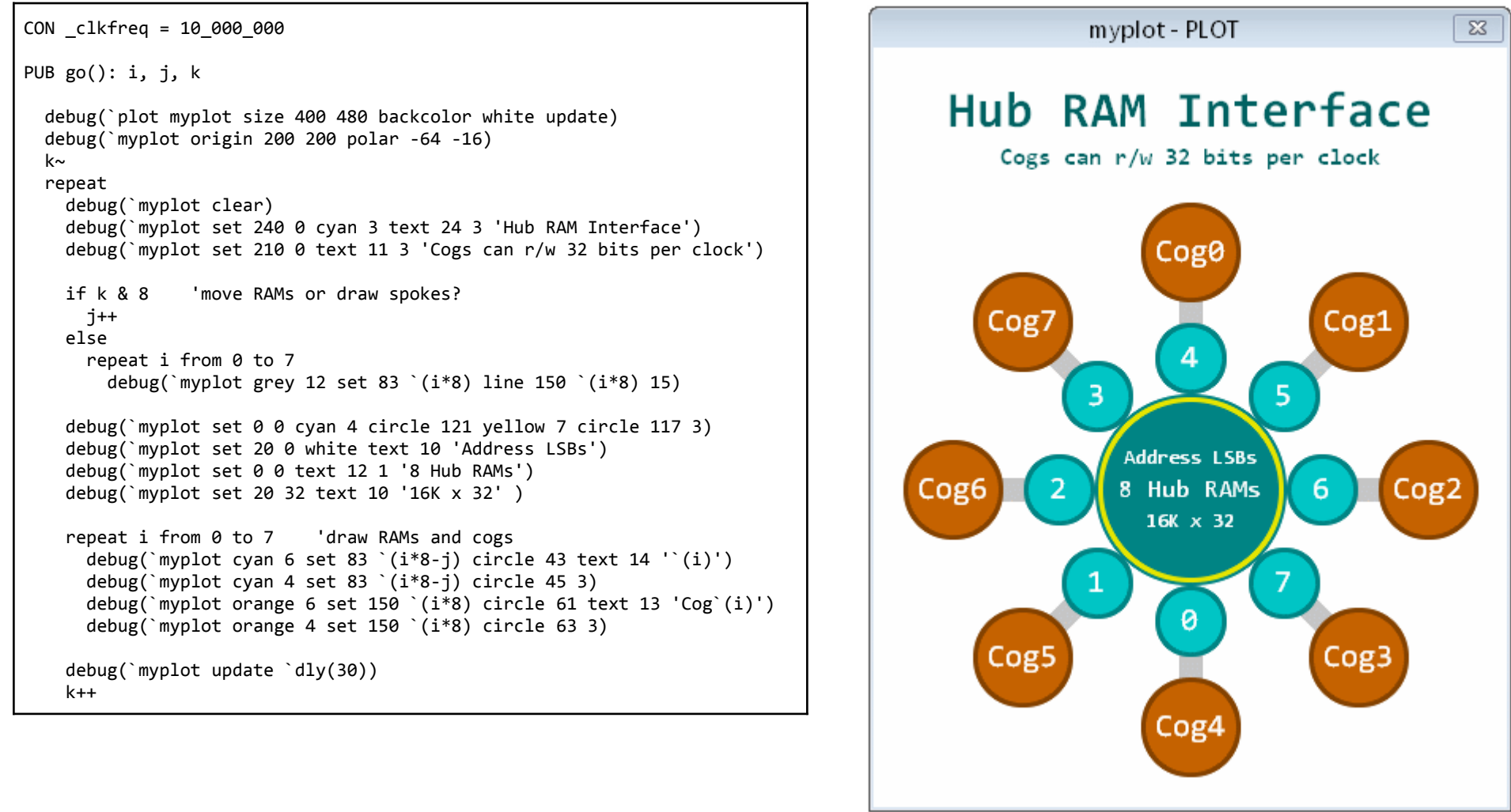


SPECTRO Instantiation	Description	Default
TITLE 'string'	Set the window caption to 'string'.	<none>
POS left top	Set the window position.	0, 0
SAMPLES 4_to_2048 {first {last}}	Set the 2 ⁿ number of FFT input points, plus the first and last result values to display (defines display height).	512, 0, 255
DEPTH 1_to_2048	Set the number of vertical-line FFT results to display (defines the display width).	256
MAG 0_to_11	Set the magnification factor (2 ⁿ , n = 0..11).	0
RANGE saturation_power	Set the power level at which pixel brightness saturates.	\$7FFFFFFF
RATE 1_to_2048	Set the number of samples before each display update.	SAMPLES / 8
TRACE 0_to_15	Set the trace pattern (see TRACE animation in BITMAP Display).	15 (right, up, scroll)
DOTSIZE width_and_height {height}	Set the spectrograph pixel-width and pixel-height together, or set them independently.	1, 1
luma_or_hsv {color_or_phase}	Set the color scheme to LUMA8(W)(X) with color *, or HSV16(W)(X) with 0..255 phase-coloring offset.	LUMA8X ORANGE
LOGSCALE	Set log-scale mode to magnify low-level results.	<off>
packed_data_mode	Enable packed-data mode. See description at end of this section.	<none>
SPECTRO Feeding	Description	Default
data	Numerical data is fed into a sliding Hanning window from which the FFT computes power and phase.	
CLEAR	Clear the sample buffer and display, wait for new data.	
SAVE {WINDOW} 'filename'	Save a bitmap file (.bmp) of either the entire window or just the display area.	
CLOSE	Close the window.	

* Color is ORANGE / BLUE / GREEN / CYAN / RED / MAGENTA / YELLOW / GREY.

Below, a SPECTRO display was fed ADC samples from a pin attached to a microphone. This is what verbally counting from "1" to "10" looks like, spectrally. The "1" is on the left and the "10" is on the right. The vertical distance between horizontal trend lines is glottal pitch. The larger brightness trends are vocal formants. This gives some idea of how our ears perceive speech:





PLOT Instantiation	Description	Default
TITLE 'string'	Set the window caption to 'string'.	<none>
POS left top	Set the window position.	0, 0
SIZE width height	Set the display width (32..2048) and height (32..2048).	256, 256
DOTSIZE width_and_height {height}	Set the display pixel-width and pixel-height together, or set them independently.	1, 1
lut1_to_rgb24	Set the color mode.	RGB24
LUTCOLORS rgb24 rgb24 ...	For LUT1..LUT8 color modes, load the LUT with rgb24 colors. Use HEX_LONG_ARRAY_ to load colors.	default colors 0..7
BACKCOLOR color	Set the background color according to the current color mode. *	BLACK
UPDATE	Set UPDATE mode. The display will only be updated when fed an 'UPDATE' command.	automatic update
PLOT Feeding	Description	Default
lut1_to_rgb24	Set color mode.	rgb24
LUTCOLORS rgb24 rgb24 ...	For LUT1..LUT8 color modes, load the LUT with rgb24 colors. Use HEX_LONG_ARRAY_ to load values.	default colors 0..7
BACKCOLOR color	Set the background color according to the current color mode. *	BLACK
COLOR color	Set the drawing color according to the current color mode. Use just before TEXT to change text color. *	CYAN
BLACK/WHITE or ORANGE/BLUE/GREEN/CYAN/RED/MAGENTA/YELLOW/GREY {brightness}	Set the drawing color and optional 0..15 brightness for ORANGE..GREY colors (default is 8).	CYAN
OPACITY level	Set the opacity level for DOT, LINE, CIRCLE, OVAL, BOX, and OBOX drawing. 0..255 = clear..opaque.	255
PRECISE	Toggle precise mode, where line size and (x,y) for DOT and LINE are expressed in 256ths of a pixel.	disabled
LINESIZE size	Set the line size in pixels for DOT and LINE drawing.	1
ORIGIN {x_pos y_pos}	Set the origin point to cartesian (x_pos, y_pos) or to the current (x, y) if no values are specified.	0, 0
SET x y	Set the drawing position to (x, y). After LINE, the endpoint becomes the new drawing position.	
DOT {linesize {opacity}}	Draw a dot at the current position with optional LINESIZE and OPACITY overrides.	
LINE x y {linesize {opacity}}	Draw a line from the current position to (x,y) with optional LINESIZE and OPACITY overrides.	
CIRCLE diameter {linesize {opacity}}	Draw a circle around the current position with optional line size (none/0 = solid) and OPACITY override.	
OVAL width height {linesize {opacity}}	Draw an oval around the current position with optional line size (none/0 = solid) and OPACITY override.	
BOX width height {linesize {opacity}}	Draw a box around the current position with optional line size (none/0 = solid) and OPACITY override..	
OBOX width height x_radius y_radius {linesize {opacity}}	Draw a rounded box around the current position with width, height, x and y radii, and optional line size (none/0 = solid) and OPACITY override.	
TEXTSIZE size	Set the text size (6..200).	10
TEXTSTYLE style_YYXXUIWW	Set the text style to %YYXXUIWW: %YY is vertical justification: %00 = middle, %10 = bottom, %11 = top.	%00000001

	%XX is horizontal justification: %00 = middle, %10 = right, %11 = left. %U is underline: %1 = underline. %I is italic: %1 = italic. %WW is weight: %00 = light, %01 = normal, %10 = bold, and %11 = heavy.	
TEXTANGLE angle	Set the text angle. In cartesian mode, the angle is in degrees.	0
TEXT {size {style {angle}}}'text'	Draw text with overrides for size, style, and angle. To change text color, declare a color just before TEXT.	
POLAR {twopi {offset}}	Set polar mode, twopi value, and offset. For example, POLAR -12 -3 would be like a clock face. For a twopi value of \$100000000 or -\$100000000, use 0 or -1. In polar mode, (x, y) coordinates are interpreted as (length, angle).	\$100000000, 0
CARTESIAN	Set cartesian mode. This is the default mode.	
CLEAR	Clear the plot to the background color.	
UPDATE	Update the window with the current plot. Used in UPDATE mode.	
SAVE {WINDOW} 'filename'	Save a bitmap file (.bmp) of either the entire window or just the display area.	
CLOSE	Close the window.	

* Color is a modal value, else BLACK / WHITE or ORANGE / BLUE / GREEN / CYAN / RED / MAGENTA / YELLOW / GREY followed by an optional 0..15 for brightness (default is 8).

TERM Display

Terminal for displaying text

```

CON _clkfreq = 10_000_000

PUB go() | i

  debug(`TERM MyTerm SIZE 9 1 TEXTSIZE 40)
  repeat
    repeat i from 50 to 60
      debug(`MyTerm 1 'Temp = `(i)')
      waitms(500)

```

TERM Instantiation	Description	Default
TITLE 'string'	Set the window caption to 'string'.	<none>
POS left top	Set the window position.	0, 0
SIZE columns rows	Set the number of terminal columns (1..256) and terminal rows (1..256).	40, 20
TEXTSIZE size	Set the terminal text size (6..200).	editor text size
COLOR text_color back_color ...	Set text-color and background-color combos #0..#3. *	default colors
BACKCOLOR color	Set the display background color. *	BLACK
UPDATE	Set UPDATE mode. The display will only be updated when fed an 'UPDATE' command.	automatic update
TERM Feeding	Description	Default
character	0 = Clear terminal display and home cursor. 1 = Home cursor. 2 = Set column to next character value. 3 = Set row to next character value. 4 = Select color combo #0. 5 = Select color combo #1. 6 = Select color combo #2. 7 = Select color combo #3. 8 = Backspace. 9 = Tab to next 8th column. 13+10 or 13 or 10 = New line. 32..255 = Printable character.	
'string'	Print string.	
CLEAR	Clear the display to the background color.	
UPDATE	Update the window with the current text screen. Used in UPDATE mode.	
SAVE {WINDOW} 'filename'	Save a bitmap file (.bmp) of either the entire window or just the display area.	
CLOSE	Close the window.	

* Color is a modal value, else BLACK / WHITE or ORANGE / BLUE / GREEN / CYAN / RED / MAGENTA / YELLOW / GREY followed by an optional 0..15 for brightness (default is 8).